Application Servers

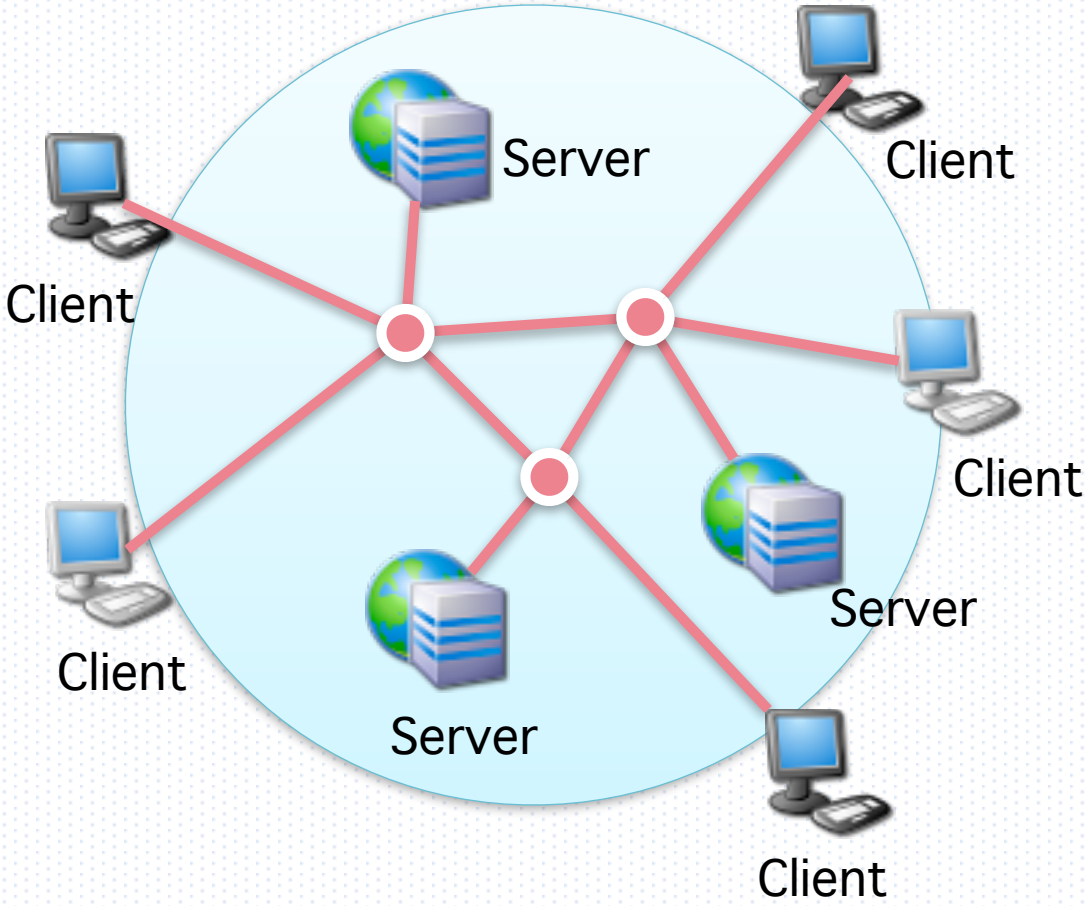
Ehsan Khamespanah

Spring 2026

What We Learn

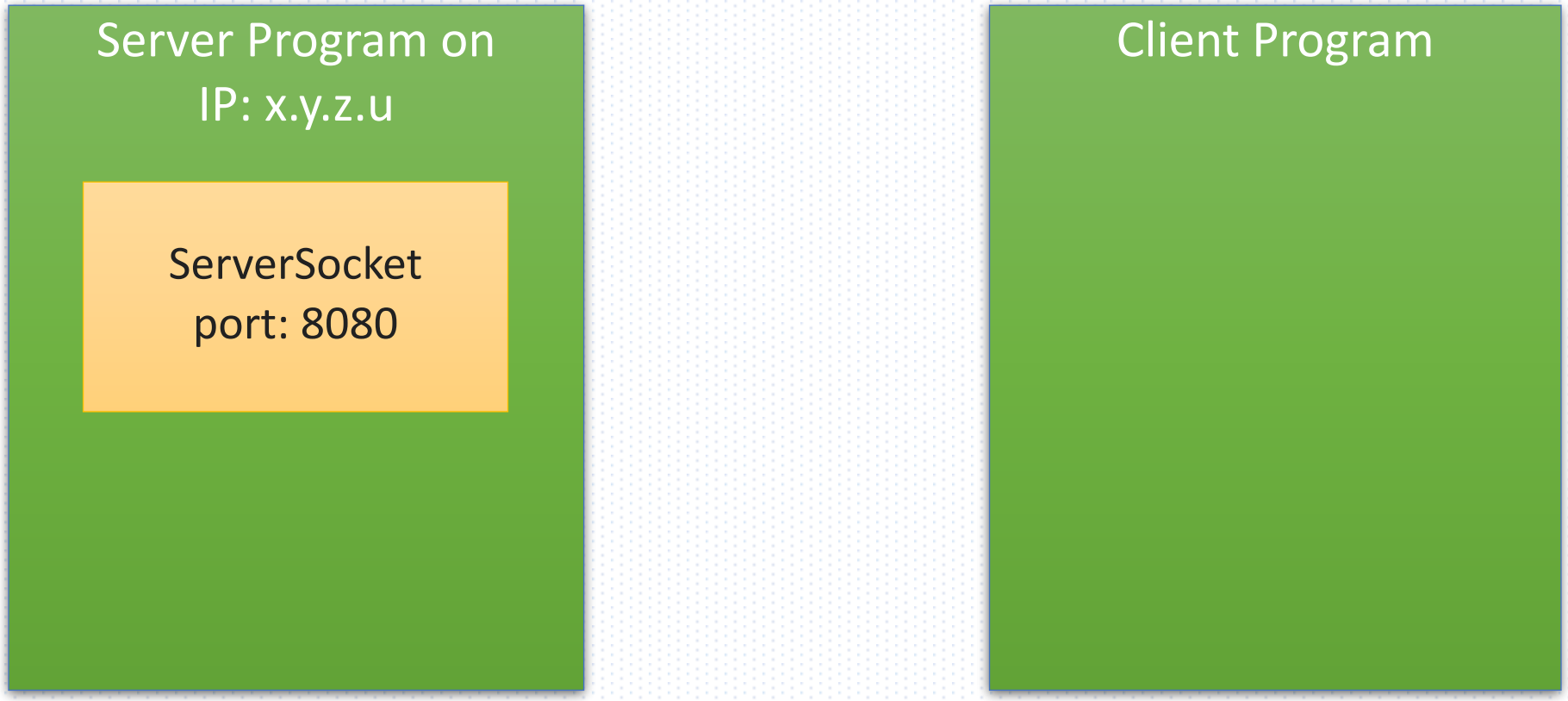
- Web Basics
  - Computer Network, HTTP, HTML basics
- Java IO Mechanism
- How to develop a Java-Based simple web application

Web Severs and Clients



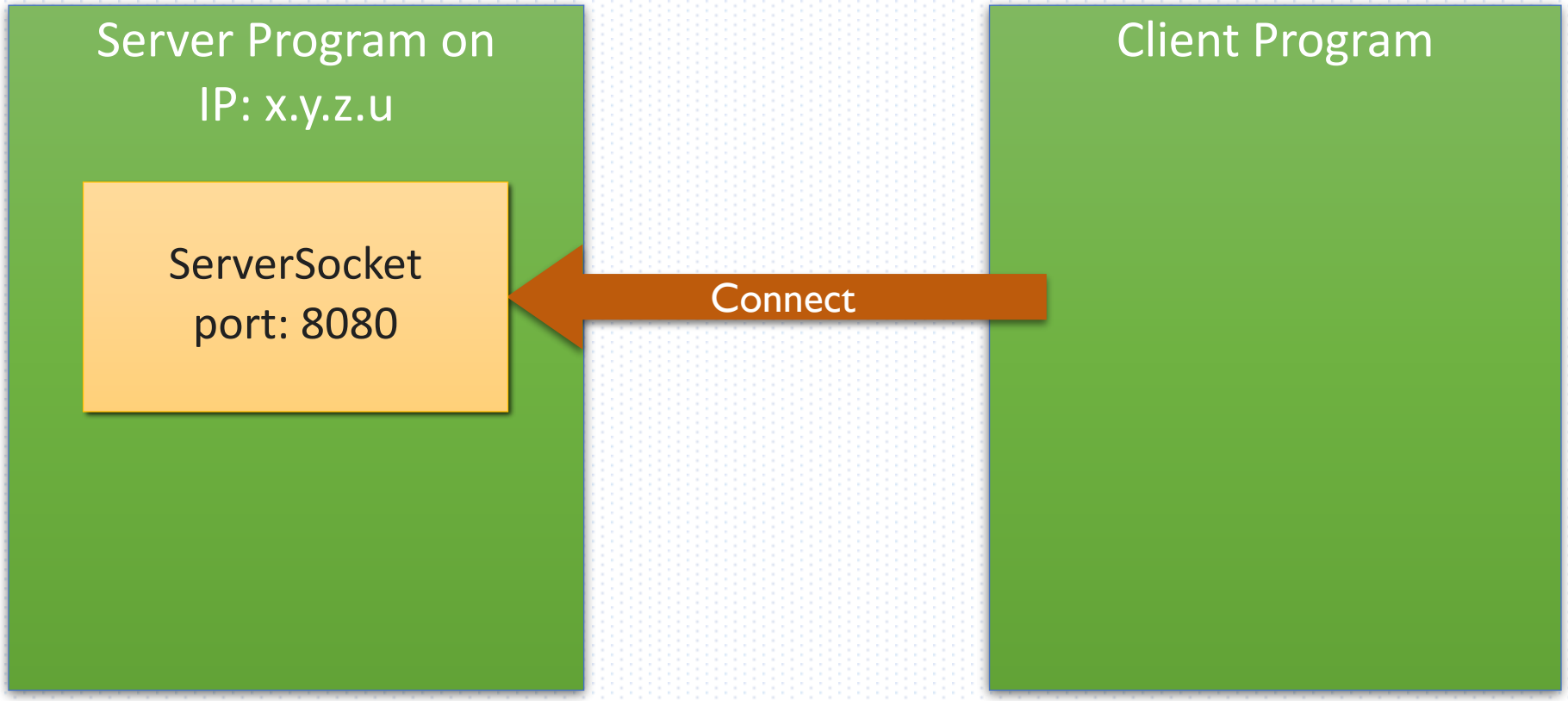
Socket Programming in Java

- ServerSocket and Socket classes
- Opens a stream between two entities



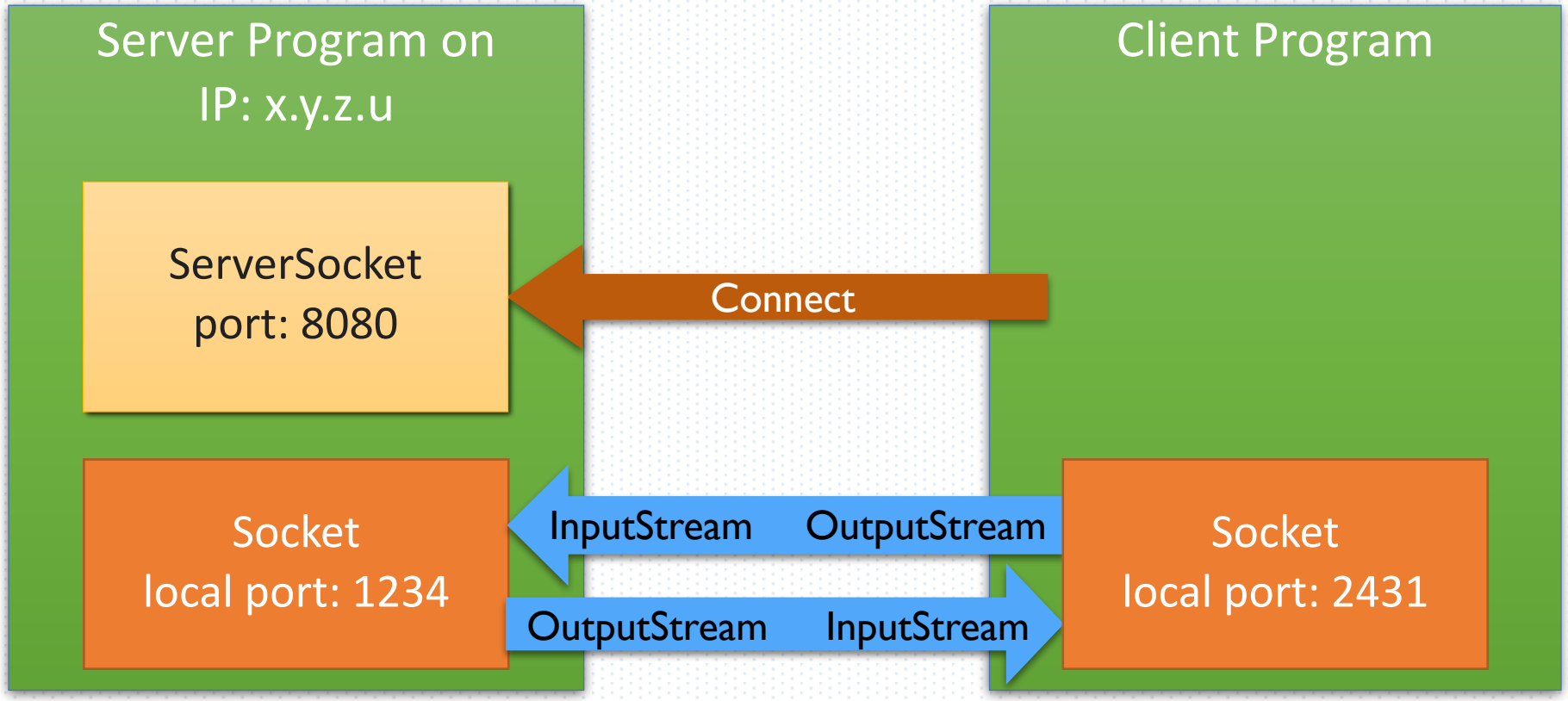
Socket Programming in Java

- ServerSocket and Socket classes
- Opens a stream between two entities



Socket Programming in Java

- ServerSocket and Socket classes
- Opens a stream between two entities



Input and Output

- Looking into input and output in an abstract way
- Input: Reading data from a source
  - Source: anything that we can read data items from
- Output:Writing data to a sink
  - Sink: anything that we can write data items to it

Streams

- Abstract class representing a data source/sink
- InputStream:
  - Represents a source
  - Provides read() methods
- OutputStream:
  - Represent a sink
  - Provides write() methods

Kind of Streams

- Streams are categorized according to type of source or sink they represent
  - Files, Arrays of bytes, Strings, Pipes, Internet connections
- InputStream example

```
public void echo(InputStream in) throws IOException {
    int b;
    while ((b = in.read()) != -1)
        System.out.print((char)b);
}
```

Readers and Writers

- A separate hierarchy but pretty similar to streams
- Added from Java 1.1 to handle Unicode IO
  - Reader is similar to InputStream
  - Writer is similar to OutputStream
  - Similar usage of filters

```
public void echoLn(Reader in) throws IOException {
    LineNumberReader st = new LineNumberReader(in);
    String line;
    while ((line = st.readLine()) != null)
        System.out.println(st.getLineNumber() + ":" + line);
}
```

Streams to Readers

- To read from a stream as a reader, use InputStreamReader
- OutputStreamWriter is used for output
- Reverse conversion is invalid

```
public void echoLn(Reader in) throws IOException {
    LineNumberReader st = new LineNumberReader(in);
    String line;
    while ((line = st.readLine()) != null)
        System.out.println(st.getLineNumber() + ":" + line);
}
```

```
Reader r = new InputStreamReader(System.in);
echoLn(r);
```

Demo

Input Stream Classes

- ByteArrayInputStream
  - Arrays of bytes
- FileInputStream
  - Files
- StringBufferInputStream
  - Strings
- ObjectInputStream
  - Serialized objects

Adding Features to Streams

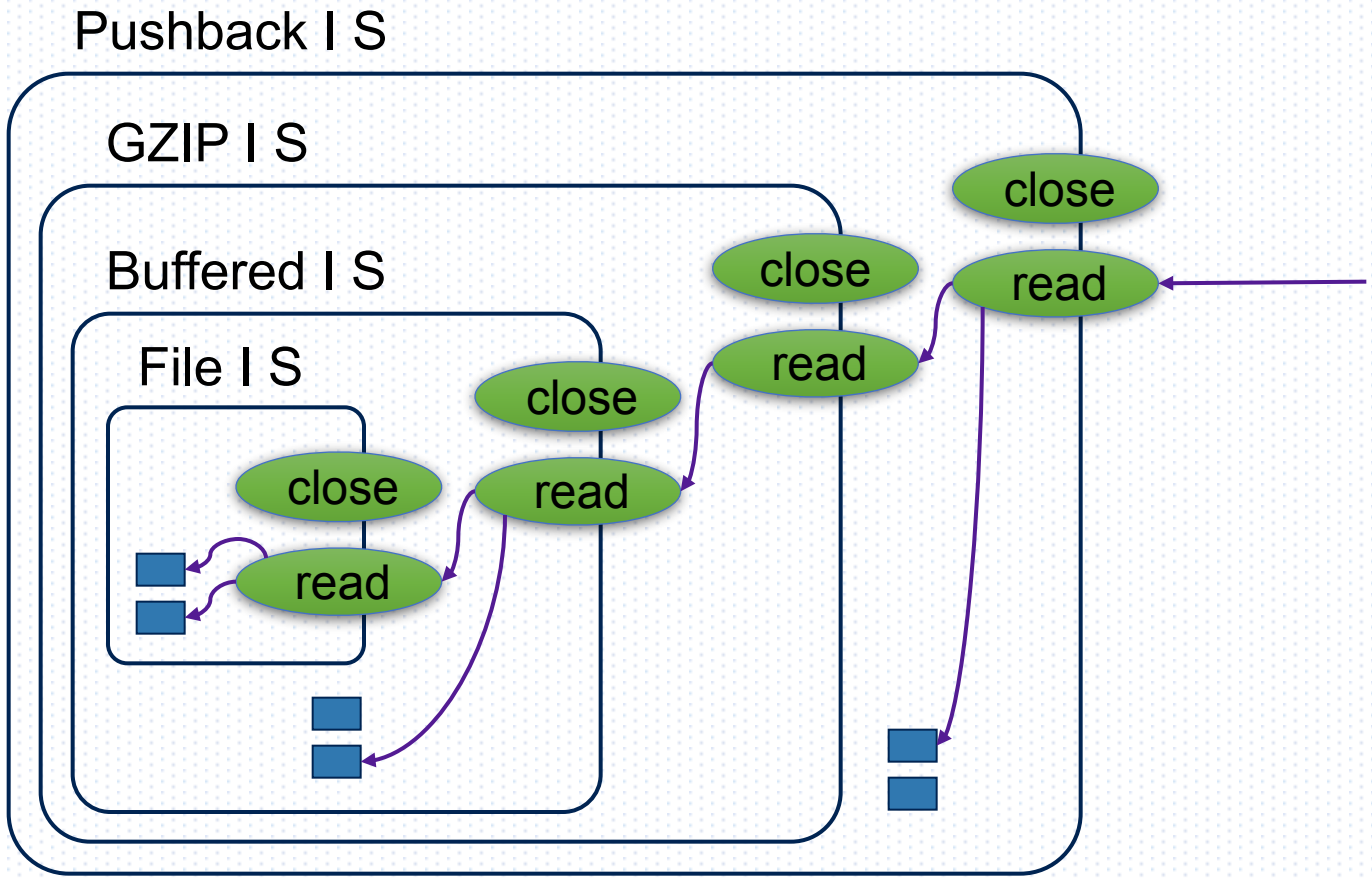
- Adding various features to streams such as:
  - Compression
  - Line-numbering
  - Buffering
  - Push-back
- Combining these features with every stream causes inheritance tree explosion!
  - CompressedFileInputStream
  - BufferedFileInputStream
  - CompressedBufferedFileInputStream, ...

Filter Streams

- Wrap around other streams
- Adding features to them
- Without changing the interface
- Such as:
  - BufferedInputStream
  - LineNumberInputStream
  - PushbackInputStream
  - DataInputStream
  - ZipInputStream

Filter Example

- We want to read
  - from a file
  - which is compressed
  - with buffering mechanism
  - with ability to pushback data into it
- We use one filter for each



Using Filters

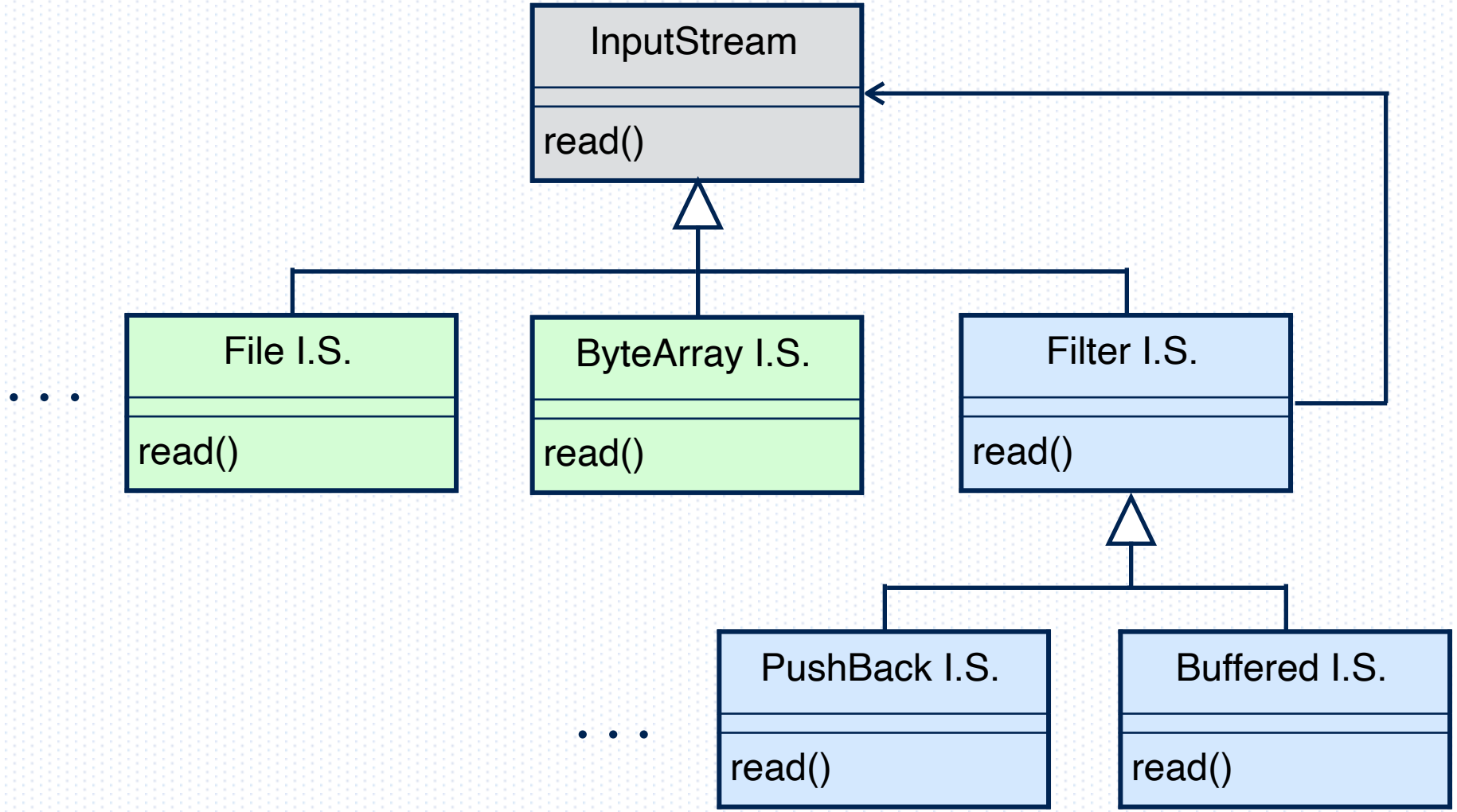
```
PushbackInputStream in = new PushbackInputStream(
    new GZIPInputStream(
        new BufferedInputStream(
            new FileInputStream("in.dat"))));
in.pushback(89);
in.close();
```

- After construction, in can be used as an ordinary input stream
- Additional methods for pushback is available

Decorator Pattern

- All filters are subclasses of FilterInputStream
- Which is a InputStream itself
- Takes an InputStream upon construction
- And filters data going through it
- The interface is not changed
- Since the filter is also an Input Stream
- This way to define classes to add features is called the Decorator pattern

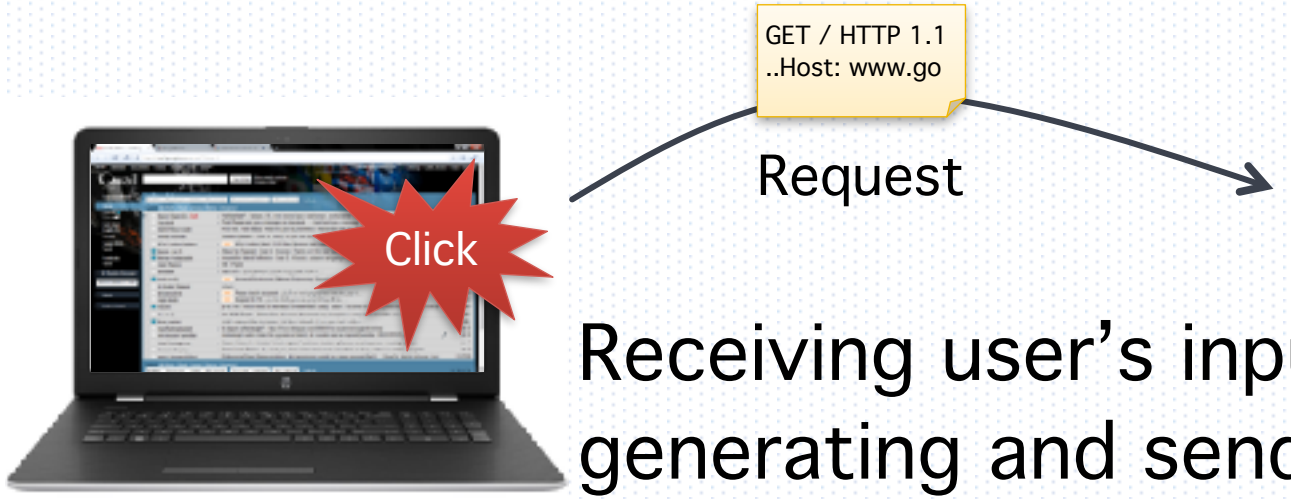
The Decorator Pattern



What Does the Clients Do?



What Does the Clients Do?

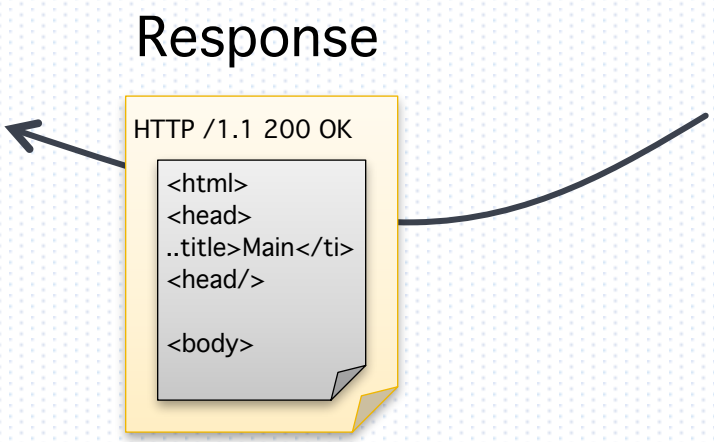


Receiving user's input, generating and sending request to the server

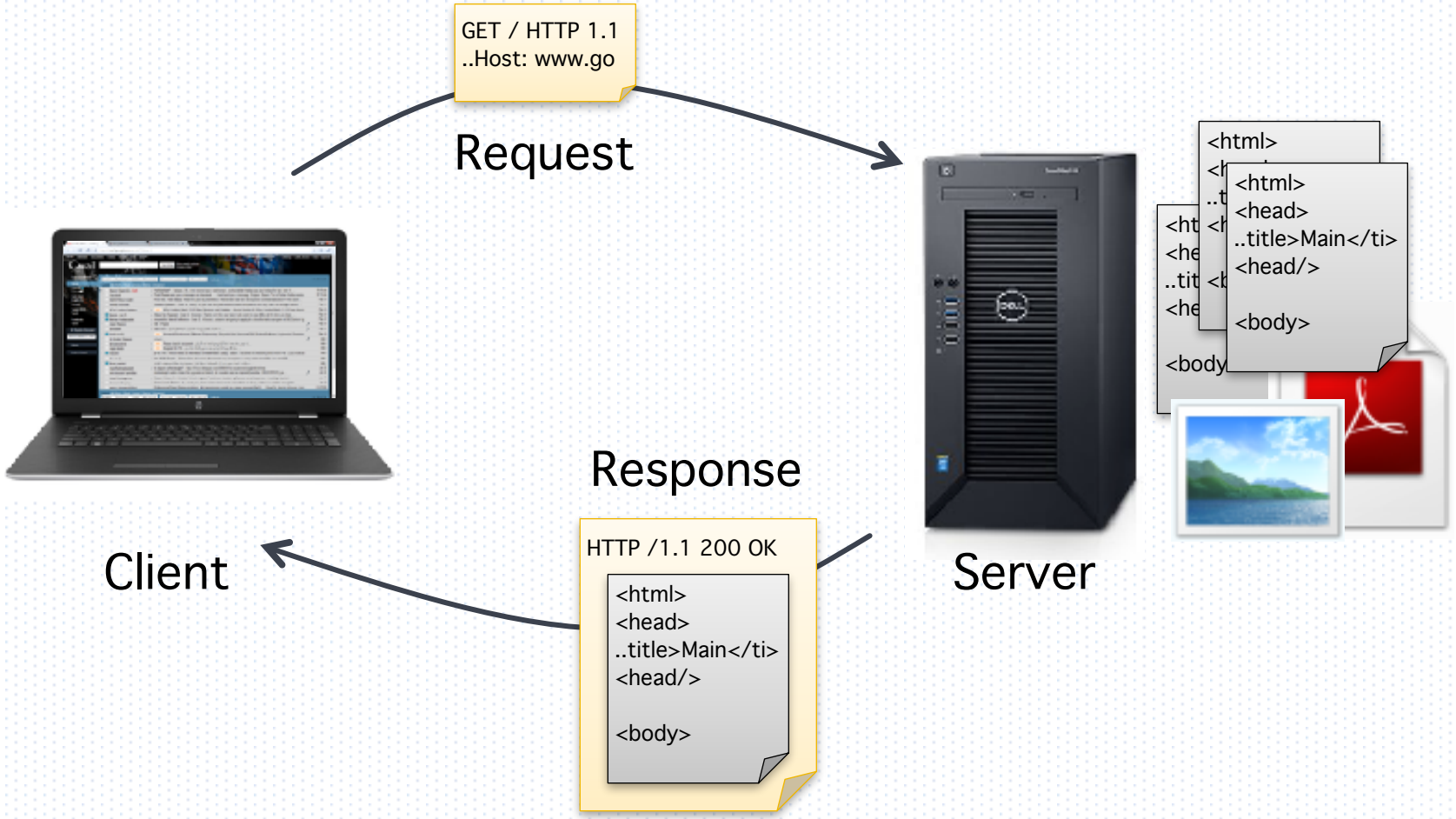
What Does the Clients Do?



Receiving and rendering the server response



What Does the Web Servers Do?



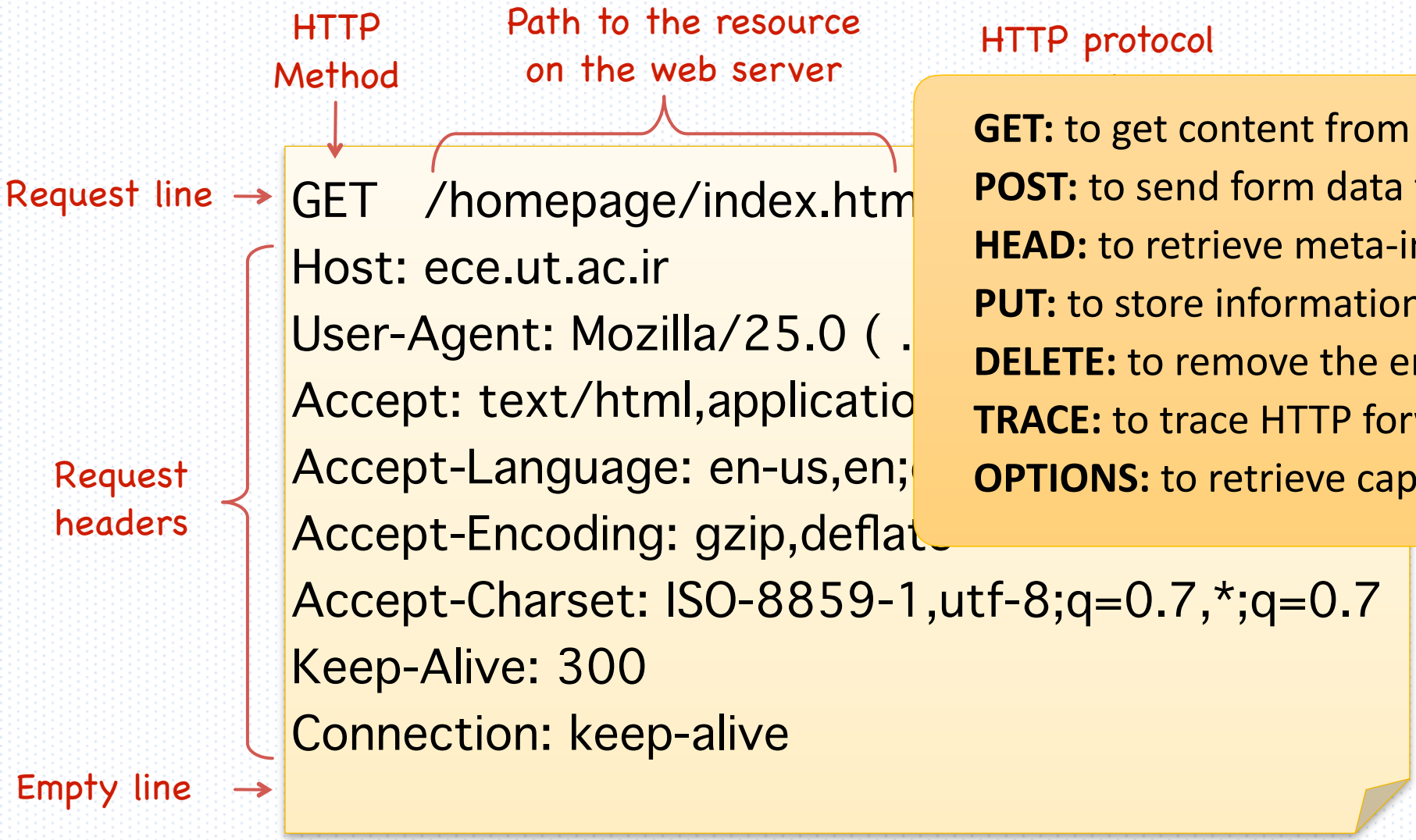
Protocols Make This Scenario Possible

- Hypertext Transfer Protocol (HTTP) is the protocol the server and the client communicate.
- Hypertext Markup Language (HTML) tells the browser how to display the content to the user.

HTTP Request

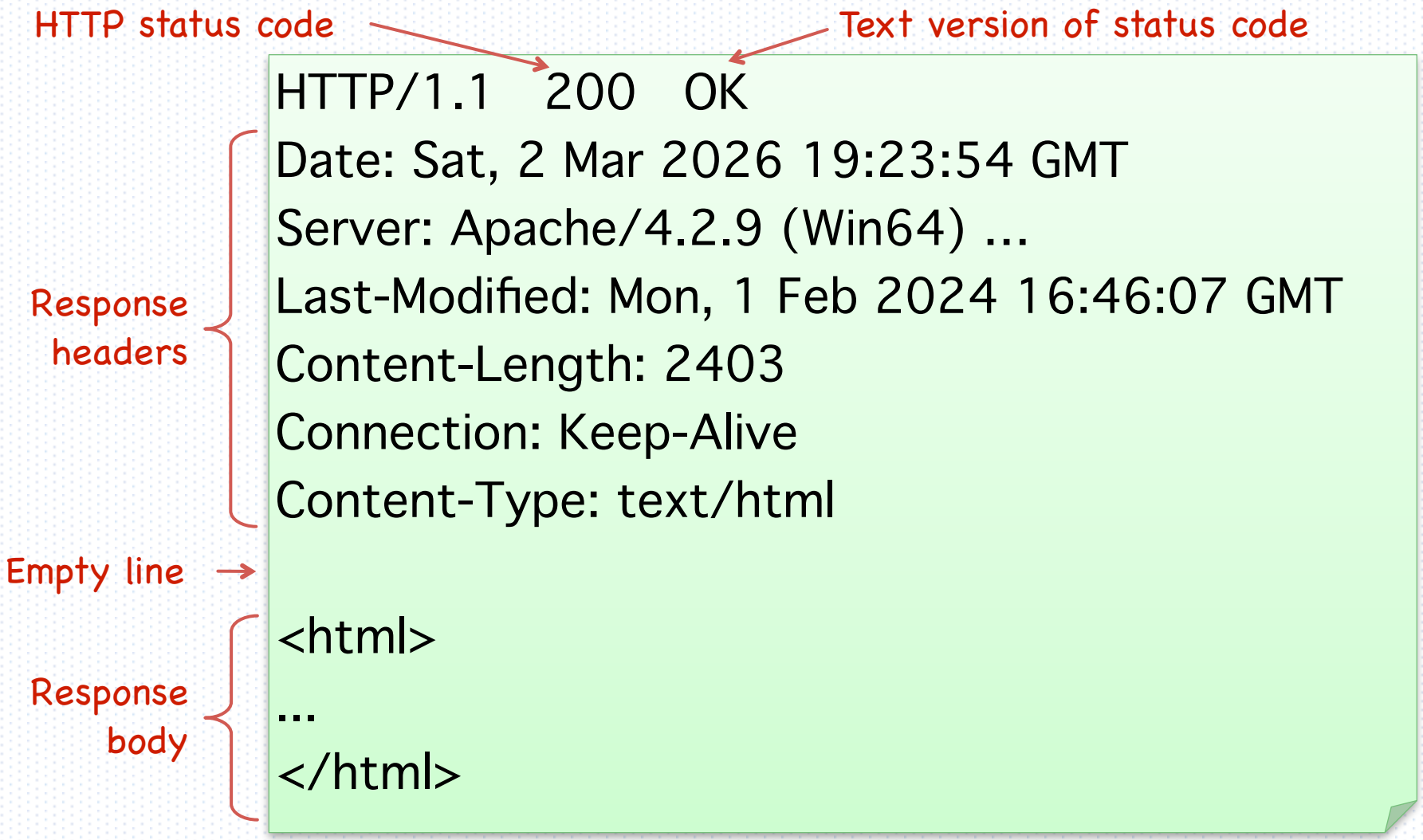


HTTP Request

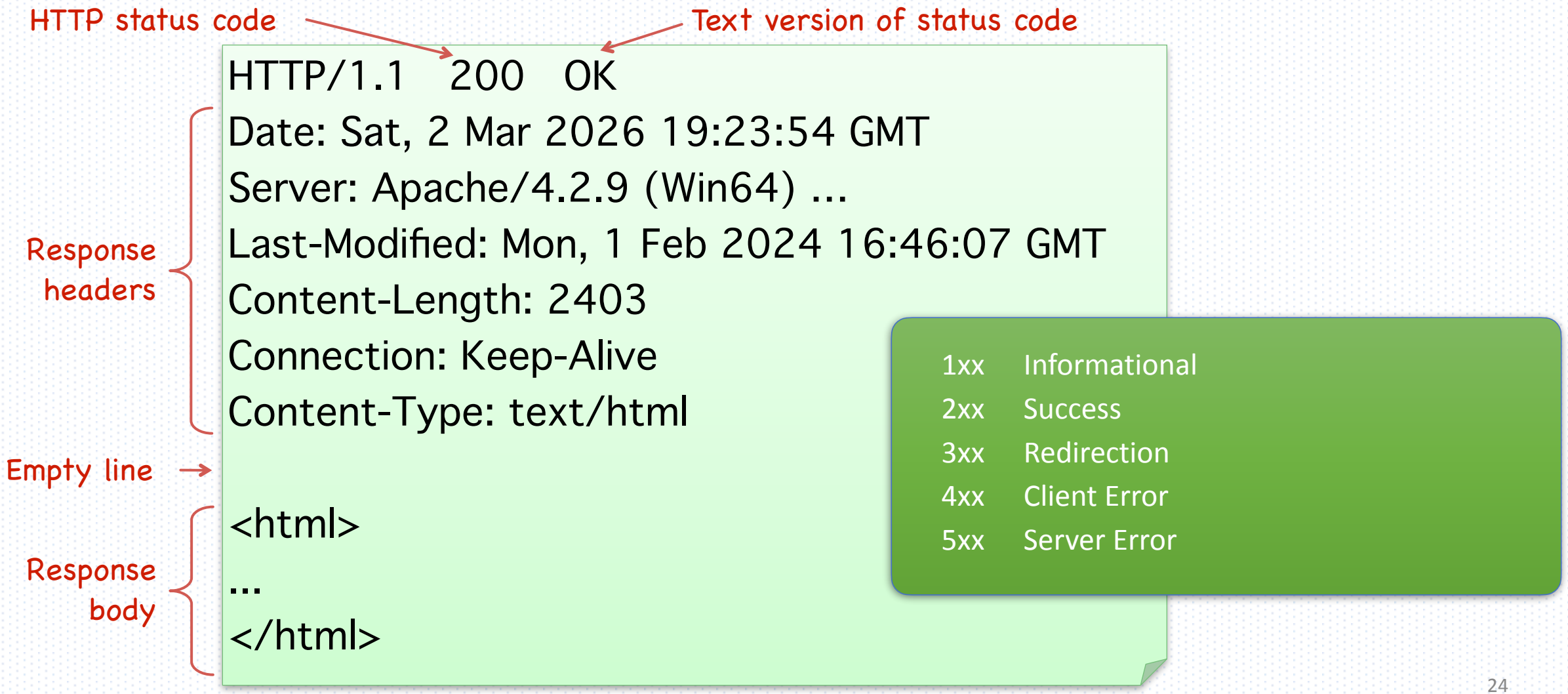


GET: to get content from the server
POST: to send form data to the server
HEAD: to retrieve meta-information
PUT: to store information under the URL
DELETE: to remove the entity under the URL
TRACE: to trace HTTP forwarding
OPTIONS: to retrieve capabilities of the server

HTTP Response

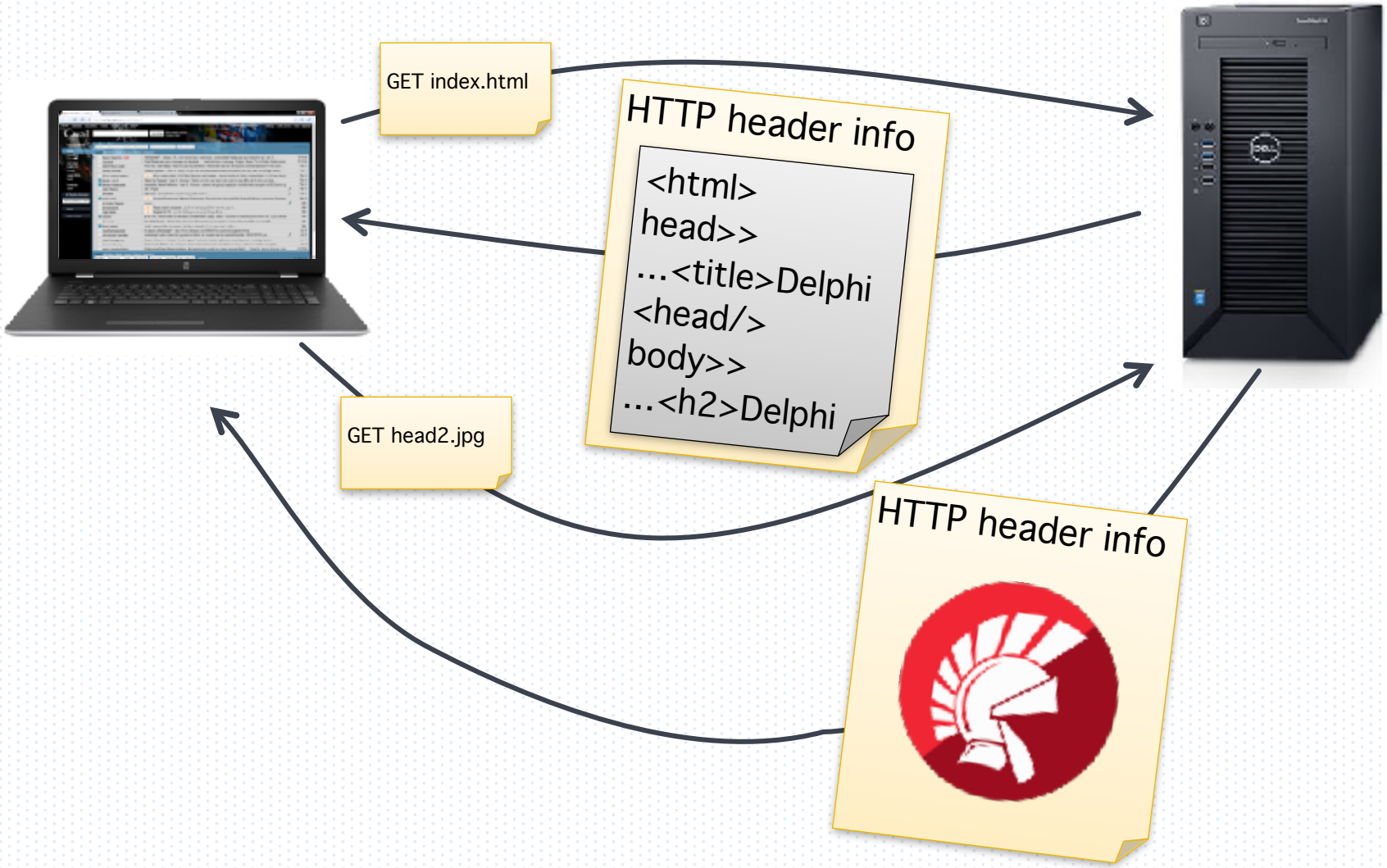


HTTP Response



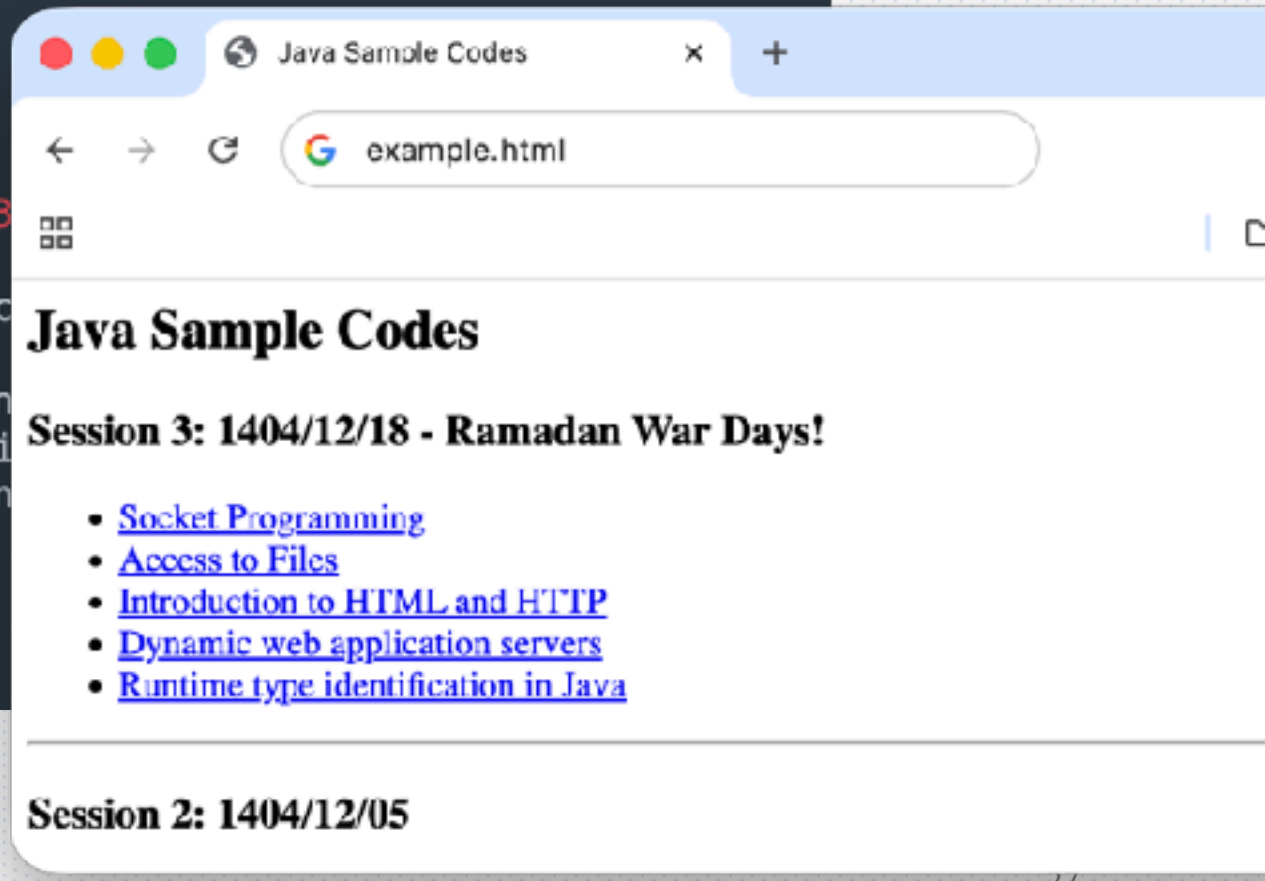
Demo

Complex Pages



Rendering HTML

```
1  <html>
2  <head>
3  <title>Java Sample Codes</title>
4  </head>
5
6  <body>
7  <h2>Java Sample Codes</h2>
8
9  <h3>Session 3: 1404/12/18 - Ramadan War Days!</h3
10 <ul>
11   <li><a href="java/socket-programming.html"> Soc
12   <li><a href="java/files.html"> Access to Files
13   <li><a href="java/HTTP-HTML.html"> Introduction
14   <li><a href="java/dynamic-web-app.html"> Dynami
15   <li><a href="java/RTTI.html"> Runtime type iden
16 </ul>
17 <hr>
18
19 <h3>Session 2: 1404/12/05</h3>
```



Versions of HTTP

- **In HTTP/1.0**: a separate TCP connection to the same server for every resource request
- **In HTTP/1.1**: a TCP connection can be reused to make multiple resource requests (i.e. of HTML pages, frames, images, scripts, stylesheets, etc.)
- **HTTP/2**: a revision of previous HTTP/1.1 with these differences in order:
  - A compressed binary representation of metadata (HTTP headers)
  - Using bidirectional streams per TCP/IP connection in which HTTP requests and responses are broken down and transmitted in small packets
  - A push capability to allow server application to send data to clients whenever new data is available
- **HTTP/3**: is a revision of previous HTTP/2 in order to use QUIC + UDP transport protocols instead of TCP.

Versions of HTML

- **HTML 1.0 (1993):** One of the first versions of HTML, Provides Structuring Elements, introduced limited fonts.
- **HTML 2.0 (1995):** Standardization of HTML, introduced the concept of forms and the <table> tag for creating tabular data
- **HTML 3.2 (1997):** Upgraded Form Elements, CSS (Cascading Style Sheets) Support, Enhanced Image Features, Extended Character Set
- **HTML 4.01 (1999):** CSS Linking, introduced some new tags (e.g. “<fieldset>”, “<header>”, “<footer>”, and “<legend>”). Table Enhancement (e.g. ‘colspan’ and ‘rowspan’)
- **HTML5 (2014):** New Form Elements, Audio Tag, Semantic Tags (e.g. <figcaption>, <header>, <footer>, <section>)

Files and Directories

- `File` class in java.io package
- Represents name of a file or a set of files
  - You cannot read or write data using File
- Used for
  - Manipulating filenames
  - Working with directories
  - Checking file attributes

```
File file =  new File(".");
System.out.println(file.isDirectory());
System.out.println(Arrays.toString(file.list()));
file.renameTo(new File("newName"));
```

```
true
[bin, target, pom.xml, .classpath, .settings,
.project, src]
```

Formatted Input

- One way is to read lines and then parse items (using Regular Expressions)
- Using tokenizers is another way
  - StringTokenizer: breaking strings into tokens

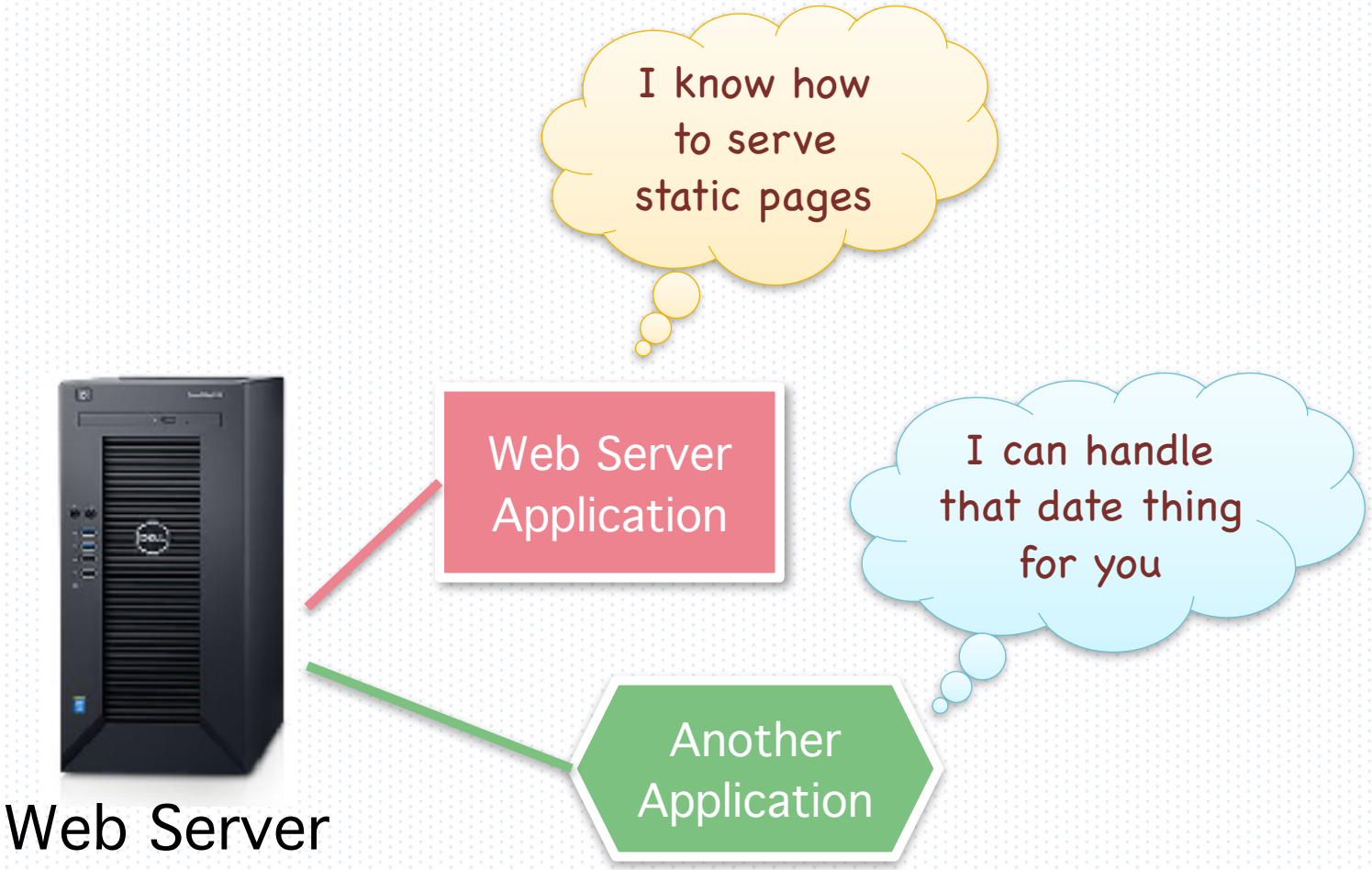
```
StringTokenizer tokenizer = new StringTokenizer(
          "A given,string-.with;delimiters", " ,-.;");
while (tokenizer.hasMoreTokens())
    System.out.println(tokenizer.nextToken());
```

```
A
given
string
with
delimiters
```

  - StreamTokenizer: reads from a stream or reader token by token

Demo

Dynamic Contents



Dynamic Contents



Web Server Machine

I know how to stat

Web Serve Applicatio

A Application

```c
#include <stdio.h>

int main() {
    printf("Content-type: text/html\n\n");

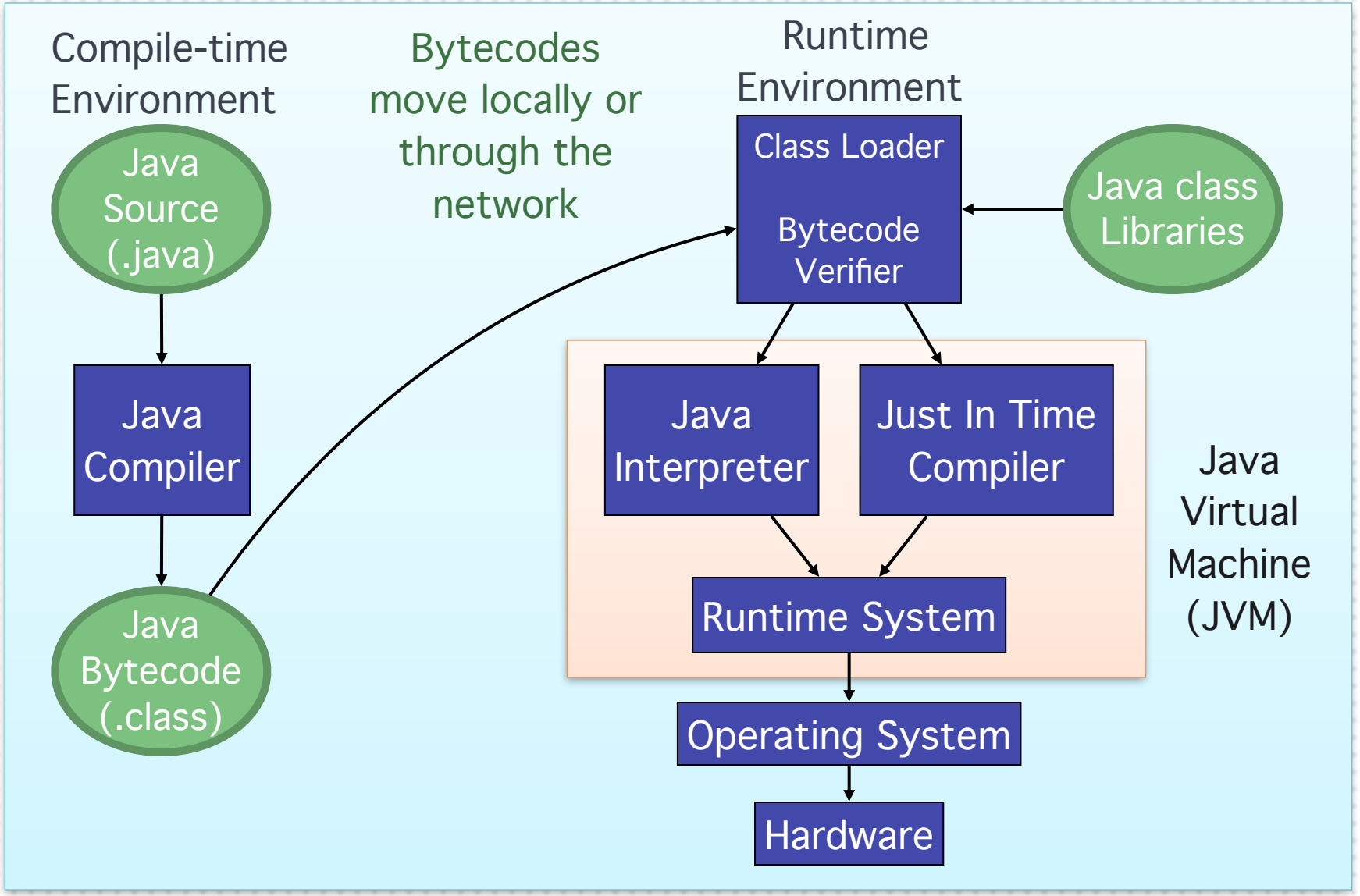
    printf("<html>\n");
    printf("<head><title>CGI Output</title></head>\n");
    printf("<body>\n");
    printf("<h1>Hello, world.</h1>\n");
    printf("</body>\n");
    printf("</html>\n");

    return 0;
}
```

Developing a Modern Web Server

- Pure socket programming is hard to do!
- We need a container to do HTTP stuff!
- We need to load classes and run their methods
  - They are not define prior to the execution time
  - Late binding of classes
- Runtime Type Identification (RTTI) mechanism of Java
  - An example: `x instanceof classX`

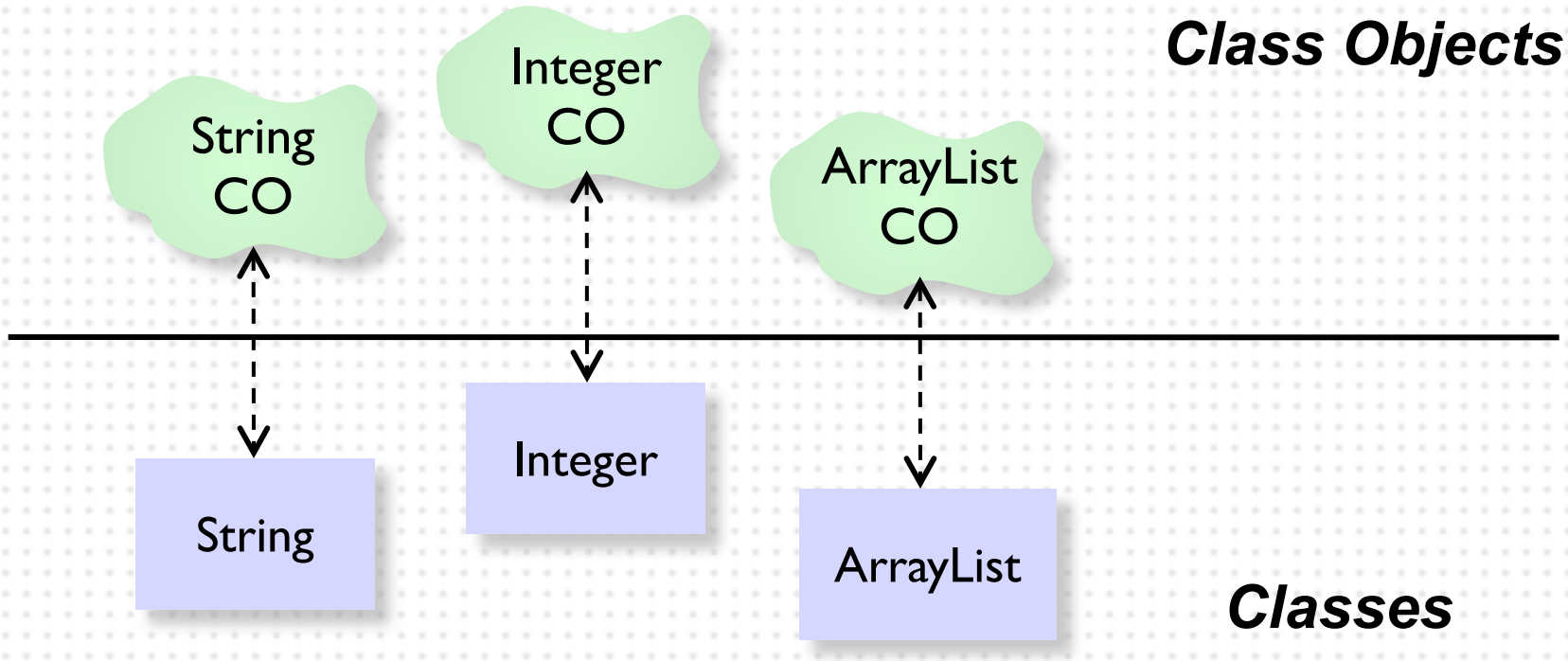
Java Technology



Reflection

String s = 10;

Class c = String.class;



Responsibilities of `class` Object

- Providing information about the class
- Its name and package
- Its superclass and interfaces implemented
- Its fields, methods, and constructors
- And more...!
- Creating instances of the class
- Getting and setting values of fields
- Invoking methods on the objects

Ways to Obtain a Class Objects

- Literally, using .class keyword

```
Class co = Circle.class;
```

- Instantiated objects know their class type

```
Shape s = new Circle(1, 1, 10);
Class co = s.getClass();
```

- Dynamically

```
Class co = Class.forName("Circle");
```

Getting Class Name

```
Shape s = new Circle(1, 1, 20);
Class co = s.getClass();
System.out.println(co.getName());

co = String.class;
System.out.println(co.getName());
```

```
Circle
java.lang.String
```

```
Scanner input = new Scanner(System.in);
System.out.print("What's the class name?");
String name = input.nextLine();
Class co;
try {
    co = Class.forName(name);
} catch (ClassNotFoundException ex) {
    co = Object.class;
}
System.out.println(co.getName());
```

Information About Methods

- Method is defined in `java.lang.reflect`
- We call `mtds[i]` a Method Object (MO)
- `getMethods()` returns inherited methods too

```
Method[] mtds = co.getDeclaredMethods();
for (int i = 0; i < mtds.length; i++)
   System.out.println(mtds[i]);
```

Information About Methods

- Method is defined in `java.lang.reflect`
- We call `mtds[i]` a Method Object (MO)
- `getMethods()` returns inherited methods too

```
Method[] mtds = co.getDeclaredMethods();
for
    S
```

```
public int java.lang.String.hashCode()
public int java.lang.String.compareTo(java.lang.Object)
public int java.lang.String.compareTo(java.lang.String)
public int java.lang.String.indexOf(int,int)
public int java.lang.String.indexOf(int)
public int java.lang.String.indexOf(java.lang.String)
public int java.lang.String.indexOf(java.lang.String,int)
```

Calling Methods

- Calling `compareTo` method
- Dealing with both the return value and parameters

```
Class co = String.class;
Method[] mtds = co.getDeclaredMethods();
Method mo = mtds[2];
String s = "ABC";
System.out.println(
    mo.invoke(s, new Object[] {"ACX"})
);
```

```
-1
```

Benefits of Reflection

- Great degree of flexibility
- Working with a class without any knowledge about it at the development time
- Suitable for: Debuggers, Class Browsers, GUI Builders, Plug-ins, Code Generators, Reflective Frameworks

The Payed Price

- No static type checking
- Potential problems:
  - The class cannot be found
  - The object does not support the method invoked
  - Wrong number/types of parameters
  - Violating access level to members
- All must be handled using exceptions
- A bit of performance penalty

Demo